التشفير والتهشير في حفظ الأسرار من التشهير

Lecture 5 — Securing Web Application Data

> **Data classification** : sorting data into levels so each level gets the protection it needs. Treating everything the same wastes effort in one direction and creates risk in the other.

Level	Example	Required protection
Public	Marketing pages, published prices	Integrity checks only
Internal	Internal docs, non-public reports	Access control and authentication
Confidential	Financial records, business strategy	Access control + **encryption at rest**
Restricted	National ID, health records, payment cards	Encryption at rest **and** in transit, strict access logging, often regulation

> **Data in transit** : moving across a network (browser ↔ server, service ↔ service) — protected mainly by **TLS**.

> **Data at rest** : sitting in storage (database, files, backups, cache).

Both need **independent** protection: HTTPS everywhere doesn't help if the customer table is stored as plain text on disk.

Encoding vs encryption vs hashing

> **Encoding** : changes data format for compatibility (Base64, URL encoding). **No key, reversible by anyone** → provides **no confidentiality**.

> **Encryption** : transforms data with a **secret key**; only the key holder can reverse it.

> **Hashing** : **one-way**, no key, no reversal; used for password storage and integrity checks. A hash alone does **not** provide authenticity.

Technique	Reversible?	Needs a key?	Typical use
Encoding	Yes, by **anyone**	No	Format compatibility (Base64, URL)
Encryption	Yes, **only with the key**	Yes	Confidentiality in transit / at rest
Hashing	**No**	No	Passwords (Week 3), integrity verification

Exam trap: "Base64-encoded password" is **not protected**. Anyone can decode it.

Symmetric and asymmetric encryption

Symmetric	Asymmetric
One shared key encrypts and decrypts	**Key pair**: public key (shared) + private key (secret)
Fast — good for large data	Computationally expensive
Example: AES	Data encrypted with the public key decrypts only with the private key

Q/ How does TLS use both ?

1. The handshake uses asymmetric cryptography **only long enough** to authenticate and agree on a shared session key (modern TLS: asymmetric for **authentication/signatures**, **ephemeral Diffie–Hellman** for key exchange).
2. Every following byte is protected with **fast symmetric** encryption.

Key management

Q/ What are the three principles of key management ?

1. **Never store the key with the data** it protects — use a **key management service (KMS)** or **HSM** / secrets manager.
2. **Rotate keys** on a schedule, so a past exposure doesn't stay valid forever.
3. **Never hard-code keys** in source code or config committed to version control — that publishes them to anyone with repository access.

Same stolen backup, two outcomes: encrypted at rest with the key stored **separately** → useless to the attacker; key next to the data → fully readable.

> **Key lifecycle** : generate → store securely → use → rotate → revoke/destroy.

Reducing exposure

> **Tokenization** : replace a sensitive value (e.g. a card number) with a **random token** that has no mathematical relationship to it; the real mapping lives in a separate, tightly controlled **vault**. A breached database yields only meaningless tokens.

> **Data minimization** : collect only what a feature really needs, and **delete it** once its purpose is fulfilled. Data never collected can't leak.

Tokenization	Encryption
Random token, no math link to the original	Ciphertext derived from the original with a key
Reverse only by looking up the vault	Reverse with the key
Common for payment cards	General confidentiality

> **Cryptographic Failures (OWASP A02:2021)** : this lecture's category.

Q/ What does A02:2021 Cryptographic Failures cover ?

1. Sending sensitive data in **plain text** instead of over TLS.
2. **Outdated or weak algorithms** such as MD5 or DES.
3. Sensitive data **unencrypted at rest**.
4. Keys **stored or generated improperly** (hard-coded, next to the data, never rotated).
5. **Predictable random numbers** where a cryptographically secure source was needed.

Nearly every item is a direct violation of a principle from this lecture.

Summary of Web Applications Security L5: Securing Web Application Data. Built only from the lecture file; check the original slides for anything you're unsure about.